

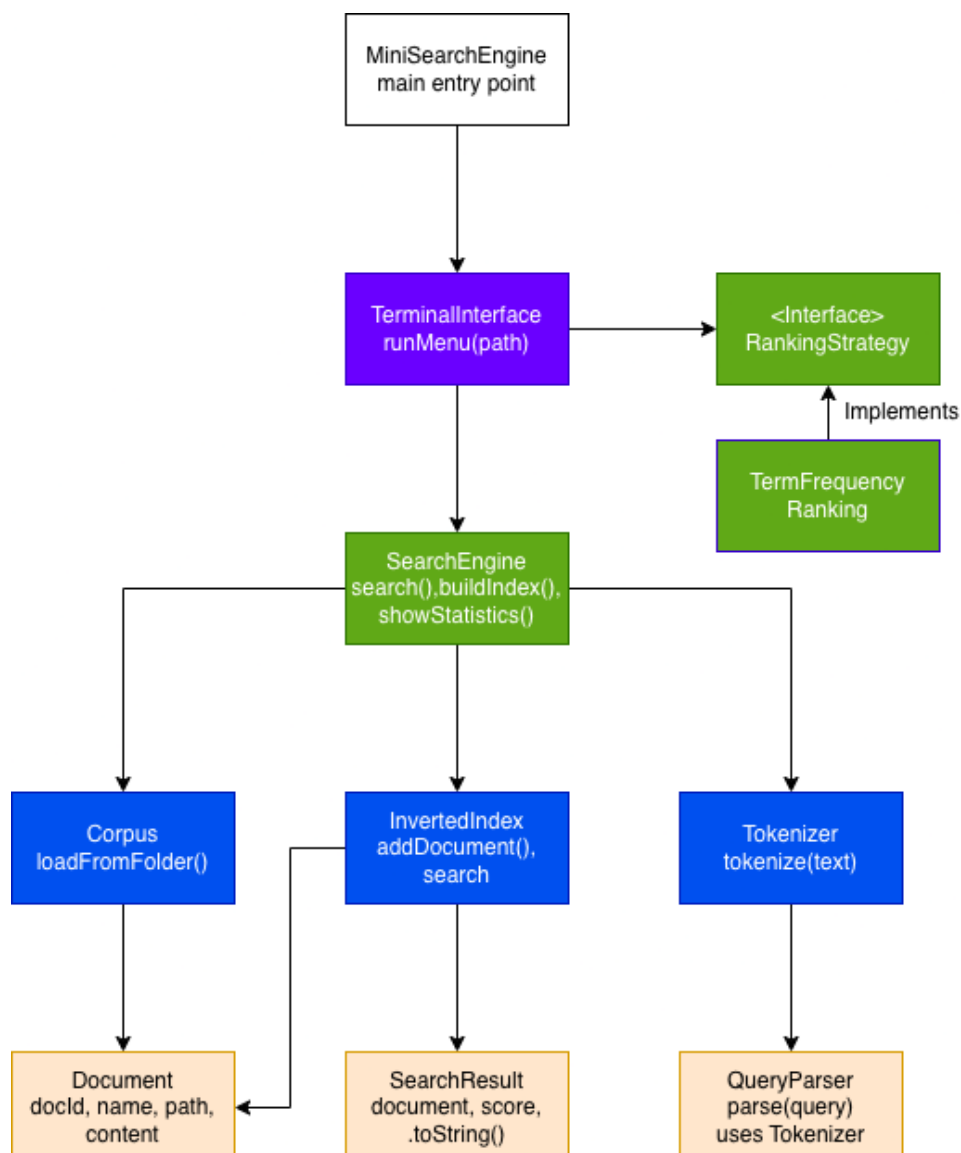
Mini Search Engine

Table of Content:

1. OOP design	2
2. Main classes and their responsibilities	3
3. Data structures	4
4. Ranking method	4
5. Limitations / future improvements	5

OOP design

- The program follows a layered OOP design architecture. The top layer contains MiniSearchEngine which acts as the entry point for the user, it validates the folder path then passes control to TerminalInterface class. It handles all user interaction, this keeps the UI completely separate from the searching logic. TerminalInterface utilizes SearchEngine, which acts as a coordinator for all other classes. While not performing indexing or searching directly, the SearchEngine coordinates with Corpus, InvertedIndex, and Tokenizer to do so. Data is represented in two classes: Document, which holds the content of each individual file, and SearchResult which pairs documents with their ranking score. QueryParser is the class responsible for interpreting user input into searchable terms, it uses Tokenizer to avoid duplicating text processing logic. All class fields are made private and are only accessible through getters. This enforces encapsulation all throughout the program. TermFrequencyRanking, implements RankingStrategy interface. This allows for ranking methods to be changed in the future without modifying SearchEngine.



Main classes and their responsibilities

- MiniSearchEngine, is the entry point of the program. It makes sure that a folder path has been provided through the command line argument and passes it on to TerminalInterface.
- TerminalInterface, manages the terminal menu for the user. It presents options to the user to choose from, and routing them based off their choice to the appropriate method in SearchEngine. Also tracking whether or not index has been built.
- SearchEngine, is the central coordinator of the program. It is where the user, loads the corpus, builds the index, performs searches, and displays the statistics. It assigns the word to Corpus, InvertedIndex, and Tokenizer.
- Corpus, is responsible for storing .txt files from the given folder as Documents. It gives each file a unique ID and stores them as Document objects along with each files name, path.
- InvertedIndex, stores the mapping from terms(keys) to the mapping of Documents(keys) and their frequencies. It allows for adding documents and getting matching documents for a term.
- Tokenizer, processes text by turning them into lowercase, removing punctuation, and splitting it into tokens. It is utilized by both SearchEngine during indexing and QueryParser during query processing.
- QueryParser, takes a user query and returns a list of tokenized terms using Tokenizer.
- Document, is a data class which represents a single text file storing its name, and path.
- SearchResult, pairs a Document to a integer score relevant to a term chosen in query, which is formatted to a string representation for display.
- RankingStrategy, interface to create different ranking strategies in a scalable and modular way.
- TermFrequencyRanking, a ranking algorithm that is implemented from RankingStrategy, scores are ranked based off size. The most frequent is most relevant.

Data structures

The main data structure in this program is an inverted index, which is implemented using a `HashMap<String, Map<Document, Integer>>`. The outer map keys are terms, and each value is an inner `HashMap`. The inner map maps each `Document` that contains the term and its frequency. A `HashMap` was chosen because it allows for $O(1)$ average-case lookup by key. This is important as the user will do a lot of searching, allowing to make query terms to documents without scanning the entire collection.

Documents are stored in `ArrayList<Document>` inside of `Corpus`. A list is a good choice as documents are loaded once and iterated over sequentially during indexing.

`SearchResults` are collected into an `ArrayList<SearchResult>` and are stored in descending order of score using `List.sort()` with a comparator. This gives the user scores ranked from most to least frequent/relevant.

Tokens are created by `Tokenizer` and are returned as `ArrayList<String>`, which are iterated over to create the index. A list of fine here since tokens are processed in order and no duplication happens at this stage. The inverted index handles the frequency counting naturally for us.

Ranking methods

The program uses term frequency scoring to rank the search results. When a query is made. Each term is searched for in the inverted index and the frequency of that term in each matching document is collected. These frequencies are combined into a score for each document, if a document contains multiple query terms their frequencies are added together to produce a total score. Documents are then sorted in descending order of their scores, so the most relevant ones appear first.

Multiple word queries are treated as OR queries. This means a document has to contain at least one query term to be appear in the results, but documents who match more terms will naturally have a higher score due to having a higher accumulated frequency count.

To give an example, a given the query "Java Inheritance", a document with "java" 5 times and "inheritance" 3 times receives a score of 8. While a document with "java" 2 times receives a score of 2 and appears lower in the results.

Limitations / future improvements

One current limitation of the design is absence of word removal. Words such as “this”, “is”, and “and” are indexed and searched like all other terms. This affects the relevance scores and adds unwanted noise with terms that don’t really show the true nature of each document. A future improvement would be to use a word filter to remove unwanted terms.

The system also lacks stemming, what this means is that words such as “running”, “run”, and “runs” are all treated as different words in the index. The issue is that these words stem from the same term “run”. Adding a stemming algorithm such as Porter Stemmer would allow to match documents containing the same words of different forms.

Finally, the system uses lazy loading for the document content. The file contents are only read from the disk when first accessed during indexing, rather than all at once during startup. A future improvement would be to cache frequently accessed documents in memory, this would improve search performance for repeatedly used queries in larger collections.